Problem Set 6

Due March 6, 2026 at 11:59 PM Pacific

Turn in your problem set by Gradescope. Include any code you write in your solutions as well. Mark down how much time you spent on the homework set.

1. **Point Cloud Alignment:** Consider two point clouds $(r_1, \ldots, r_N)$ and $(\hat{r}_1, \ldots, \hat{r}_N)$, where $r_i, \hat{r}_i \in \mathbb{R}^3$ for all i. We would like to translate and rotate the second point cloud so that it best matches with the first. Assume both clouds have the same number of points and we know which point in the first cloud corresponds to which point in the second (these assumptions greatly simplify the problem and are the focus of current research). We would like to find a rotation R and translation τ that minimizes the following loss

$$J(R, \tau) = \frac{1}{N} \sum_{i=1}^{N} ||R\hat{r}_i + \tau - r_i||_2^2$$

subject to

$$RR^T = I$$

This is a non-convex optimization problem due to the orthogonality constraint $RR^T = I$. However, we can still solve this problem optimally. Let's seek to solve this optimization problem through two ways.

(a) Solve for the optimal $\tau*$ as a function of $R, \hat{r}_i, r_i$. This should just be the solution to an ordinary least-squares problem. In terms of properties of the two point clouds, what does τ physically represent?

Solution: Before we take the gradient of J w.r.t τ, let's first show why we can do this decoupling for τ (i.e. solve for τ first, then R), which will be useful in proving optimality for part (b). Simply, the minimization of $J(R, \tau)$ is

$$\min_{R,\tau} J(R, \tau) = \min_{R} \min_{\tau} \frac{1}{N} \sum_{i=1}^{N} ||R\hat{r}_i + \tau - r_i||_2^2.$$

Namely, a minimization over all decision variables is equivalent to splitting the minimizations into nested subproblems, and the ordering of which minimization to perform first does not matter. Therefore, we are free to order the minimization as above and solve for the inner minimization first, which is a least squares problem for τ.

$$\nabla_\tau J(R, \tau) = \frac{1}{N} \sum_{i}^{N} [R\hat{r}_i + \tau - r_i] = 0$$

$$\therefore \tau^* = \frac{1}{N} \sum_{i}^{N} r_i - R\frac{1}{N} \sum_{i}^{N} \hat{r}_i$$

$$= r_c - R\hat{r}_c.$$

Physically, this makes a lot of sense. The optimal translation between the two point clouds is simply the offset between the centroid of the target point cloud r_c and the centroid of the source point cloud $\hat{r}_c$ rotated into the target's frame.

(b) Solve for the rotation R^* by treating it as another least-squares problem. In the process, also show that the rotation matrix you solved is globally optimal (i.e. there doesn't exist another rotation matrix that can have a lower loss).

Hint: The Singular Value Decomposition $A = U\Lambda V^T$, where U, V are orthogonal and Λ is diagonal, may come in handy. All matrices have an SVD decomposition.

Solution: Using the above solution for τ^*, we plug that into the minimization problem to yield

$$\begin{aligned}\min_R J(R) &= \min_R \frac{1}{N}\sum_{i=1}^{N} ||R\hat{r}_i + r_c - R\hat{r}_c - r_i||_2^2 \\ &= \min_R \frac{1}{N}\sum_{i=1}^{N} ||R\underbrace{(\hat{r}_i - \hat{r}_c)}_{\hat{q}_i} - \underbrace{(r_i - r_c)}_{q_i}||_2^2.\end{aligned}$$

We can expand this cost function out while enforcing the orthogonal constraint $R^TR = I = RR^T$

$$\begin{aligned}{}_R J(R) &=_R \frac{1}{N}\sum_{i=1}^{N} ||R\hat{q}_i - q_i||_2^2 \\ &=_R \frac{1}{N}\sum_{i=1}^{N} q_i^T q_i + \hat{q}_i^T \underbrace{R^TR}_{I}\hat{q}_i - 2q_i^T R\hat{q}_i \\ &=_R \sum_{i=1}^{N} q_i^T R\hat{q}_i \\ &=_R \sum_{i=1}^{N} trace(q_i^T R\hat{q}_i).\end{aligned}$$

where the last equality holds because $q_i^T R\hat{q}_i$ is a scalar. Let's analyze each individual term of this sum. Using the cyclic property of trace, $trace(q_i^T R\hat{q}_i) = trace(R\hat{q}_i q_i^T)$. Therefore, we are equivalently finding the argmax of $trace(RH)$, where $H = \sum_{i=0}^{N} \hat{q}_i q_i^T$ (simultaneously solving for part (c)) due the linearity of trace.

Let's assume without loss of generality that we represent H by its SVD $H = U\Lambda V^T$ (since every matrix has an SVD) and R as AVU^T. We can always write R as AVU^T because VU^T yields an orthogonal matrix by construction, and we can think of A as some correction (i.e. if VU^T is suboptimal, we can rotate it by some A to make it optimal). Rewriting our cost

$$\begin{aligned}{}_R J(R) &=_R trace(RH) \\ &=_R trace(AV\underbrace{U^TU}_{I}\Lambda V^T) \\ &=_R trace(A\underbrace{V\Lambda V^T}_{PD}).\end{aligned}$$

Now, we claim the following statement is true

$$\sum_i^N k_i^T Ak_i = trace(AK^TK) \leq trace(K^TK) = \sum_i^N k_i^T k_i$$

where k_i are the columns of K. The inequality can be shown using the Cauchy-Schwarz Inequality, namely we look at each element of the sum on the LHS and RHS. Cauchy-Schwarz states that

$$|\langle u, v\rangle|^2 \leq \langle u, u\rangle \cdot \langle v, v\rangle \Leftrightarrow |\langle u, v\rangle| \leq \sqrt{\langle u, u\rangle \cdot \langle v, v\rangle}.$$

Applying Cauchy-Schwarz to each term, we find that

$$\underbrace{k_i^T}_{u^T}\underbrace{(Ak_i)}_{v} \leq \sqrt{(k_i^Tk_i)(k_i^T\underbrace{A^TA}_{I}k_i)} = k_i^Tk_i$$

and therefore our claim is true. Since our claim is true, $trace(AK^TK) \leq trace(K^TK)$ for all orthonormal A. Then, it is clear that we just need to set $A = I$ to yield the max value. Therefore, our optimal rotation matrix is $R = AVU^T = VU^T$.

(c) Show that your answer in part (b) is equivalent to taking the SVD of a simpler 3×3 matrix $H = \sum_j^N (\hat{r} - \hat{r}_c)(r - r_c)^T$.

Remark: Part (c) is important because you will need a fast implementation of getting the rotation for Problem 3.

Solution:

This was already shown in the proof for part (b).

(d) Align two point clouds, one the ground-truth `ground-truth.ply` and the other, a rotated, translated point cloud with additive noise `transformed-associated.ply`. What's the loss $J(R_t^*, \tau_t^*)$?

Solution: The loss is **0.00749**.

(e) What's the SE(3) error between your optimized pose and the ground-truth pose (located in `gt-pose.csv`)? The CSV file stores the pose as a quaternion (x, y, z, w) and the translation (px, py, pz) in that order. Report both the rotational error (in degrees), and the translational error.

Solution: The SE(3) error can be retrieved by computing a correction transformation T_c that transforms / "corrects" the resultant world frame of the optimization problem to the true world frame as expressed by the ground-truth transformation matrix. This correction is defined as $T_c\hat{T} = T_{gt}$. The norm of the axis-angle representation of the rotational part of T_c is the rotational error, while the norm of the translation part of T_c is the translational error. The rotational error is $1.59 \cdot 10^{-14}$ deg. The translational error is $1.37 \cdot 10^{-15}$. For downsampled point clouds of 10, the error is 0.000413 deg, and 0.000131 translation.

(f) Is it possible that the program is optimally solved, but the R matrix of the solution does not correspond to a physical rotation? In which case, what does R physically do to the point cloud? (Aside: even when R is not a rotation matrix, there are methods to still extract a valid rotation).

Solution:

This is possible when the determinant of the matrix R is -1. This corresponds with a reflection of the point cloud if points in the point cloud are co-linear.

2. **Bundle Adjustment SLAM in 2D:** In this problem you will use a standard open source SLAM package, GTSAM, to perform a bundle adjustment SLAM. (Recall, bundle adjustment SLAM solves for both landmark locations and robot poses at the same time. Alternatively, Pose Graph Optimization (PGO) SLAM pre-processes raw sensor measurements in a "front end", and optimizes only robot poses in a "back end.") We will use the Python interface to the GTSAM library, which is implemented in C++. We use a sparse 2D problem setup to ensure quick optimization in Google Colab. If you choose to consider dense or 3D SLAM for your final project, you will likely not want to use Google Colab, and you would likely use a PGO style SLAM.

(a) Draw a simple factor graph by hand or with a computer program, such as PowerPoint, that meets the following criteria.

- Three pose states x_0, x_1, and x_2.
- Two landmarks, ℓ_0 and ℓ_1, marked by a 2D position.
- Four range and bearing measurements as follows:
  - $y_{0,0}^{rb}$: between pose x_0 and landmark ℓ_0
  - $y_{0,1}^{rb}$: between pose x_0 and landmark ℓ_1
  - $y_{1,1}^{rb}$: between pose x_1 and landmark ℓ_1
  - $y_{2,1}^{rb}$: between pose x_2 and landmark ℓ_1

- Odometry measurements $y^o_{i,i+1}$ between successive poses.
- Loop closure constraint as odometry-like measurement $y^{lc}_{0,2}$ between pose states x_0 and x_2.

Solution: One way to draw the graph is below. There are many ways to draw the factor graph. Color is not required, but the labelling is required.

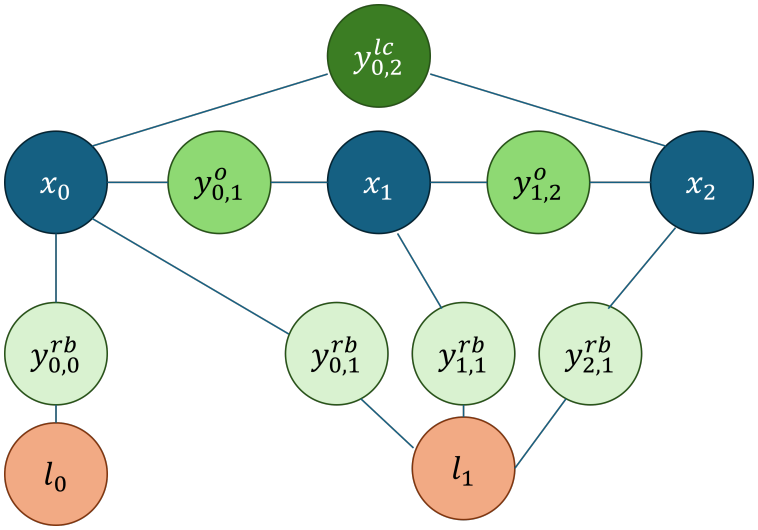


Figure 1: Simple factor graph

We now consider a more complex scenario. We will use the robot trajectory and landmarks in Figure 2. You will see three data files on Canvas. In the file `robot_history_5.csv`, we provide the ground truth history of the robot pose and measurements for a maximum sensor range of 5 meters. We will add noise and reconstruct the robot pose and landmark locations. The following steps will guide you in constructing the factor graph in GTSAM.

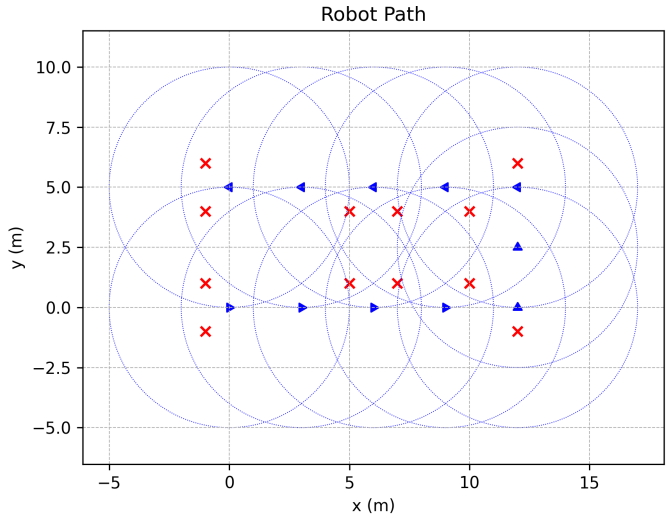


Figure 2: Robot Ground Truth Path. The landmarks are indicated as a red '×', the robot is a blue triangle oriented in the direction of travel, and the dotted blue circles indicate the maximum sensor range.

Please refer to the provided Google Colab at the link below (using your Stanford login), though you do not need to use Google Colab if you prefer to install GTSAM locally.

`https://colab.research.google.com/drive/1tJqy1jNUeGC2V235yTdWqyOElQdIclMp?usp=sharing`

(b) Verify that you can pip install GTSAM. On Google Colab, you can run `!pip install gtsam` at the top. Please read through the provided code to load and process the files. Instantiate the factor graph with:

```
graph = gtsam.NonlinearFactorGraph()
```

This is an empty factor graph. To *add* a factor to the factor graph, we will call `graph.add(...)`. To simplify the notation, GTSAM offers shorthand variables `X` and `L` for state and landmark, respectively, to organize the SLAM factors. In the provided Google Colab, we have initialized lists for `X` and `L`. The first factor we want to add is the prior factor to initialize the map at the origin. Fill in the required code based on the factor description below.

```
gtsam.PriorFactorPose2(X_start, pose, noise)
```

For `X_start`, use the zero-th element of the `X` list. For `pose`, use the `gtsam.Pose2(x, y, theta)` function specifying the origin and zero angle (i.e., aligned with the x-axis).

(c) Iterate through the provided data (`Pandas` is recommended). Use the factor type below to add the relative motion factors (i.e., from the odometry or relative state between successive states).

```
gtsam.BetweenFactorPose2(X_prev, X_curr, rel_pose, motion_noise)
```

Use the factor type below to add the bearing and range factors (i.e., from the sensor model) per each landmark in view at the given time step.

```
gtsam.BearingRangeFactor2D(X_curr, L_curr, bearing, range, noise
```

For the bearing measurement, use `gtsam.Rot2.fromDegrees(bearing_angle)`. Lastly, we will use the `BetweenFactorPose2` method to add a loop closure constraint between the second and eighth elements (zero-indexed) of the `X` list with position offset of 0 horizontally, 5 vertically, and 180 degrees in rotation. How many factors are in your factor graph (print `graph` to verify)?

Solution: We expect 60 factors, we encourage students to check if their answer makes sense from the data provided (i.e., you can count the factors directly from the data).

(d) We will initialize the values from the motion alone. Instantiate the initial estimate with

```
initial_estimate = gtsam.Values()
```

You can input an initial estimate with `initial_estimate.insert(X_or_L_var, pose_or_point)`. Be sure to use `gtsam.Pose2(x, y, theta)` for the pose and `gtsam.Point2(x, y)` for the landmark. Plot the initial estimates for `X` and `L` using the functions below:

`gtsam_plot.plot_pose2(...)` and `gtsam_plot.plot_point2(...)`.

Include the plot in your write-up.

Solution: If all the initial estimates are set up correctly, you will arrive at Figure 3.

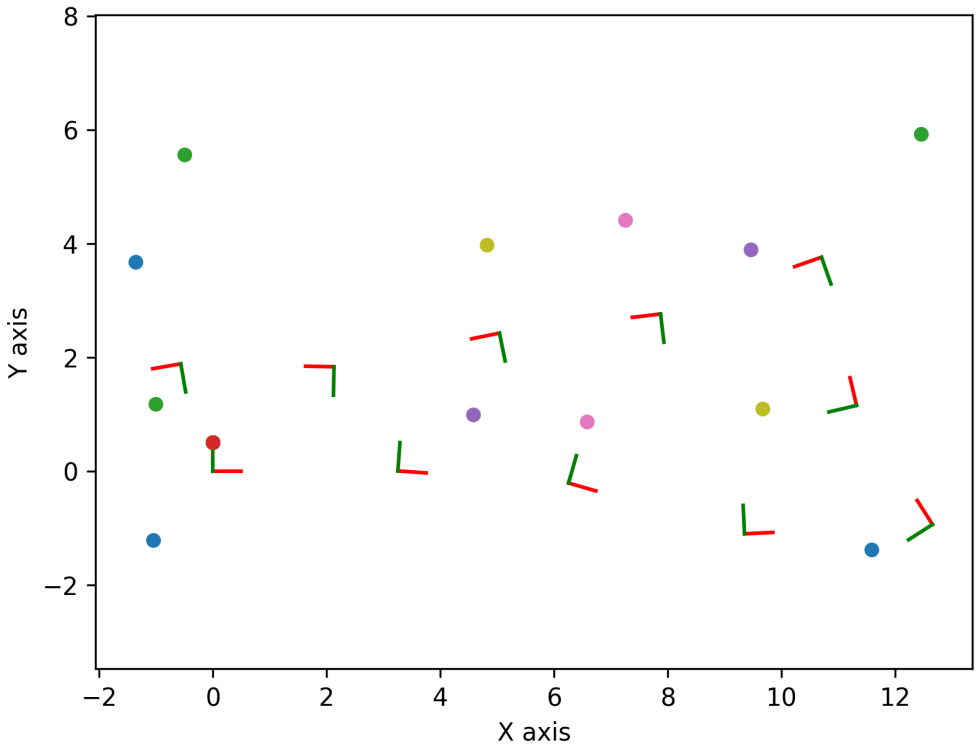


Figure 3: GTSAM Initialization

(e) Run the Levenberg Marquardt in GTSAM as listed in the Google Colab. GTSAM will provide estimates of `X` and `L` through the optimizer and will provide the marginal Gaussian distributions in `gtsam.Marginals`. Plot the optimized estimates for `X` and `L` along with the 2D Gaussian Confidence Ellipse (1-sigma). Include the plot in your write-up.

Solution: If the graph is set up correctly, you will arrive at Figure 4.

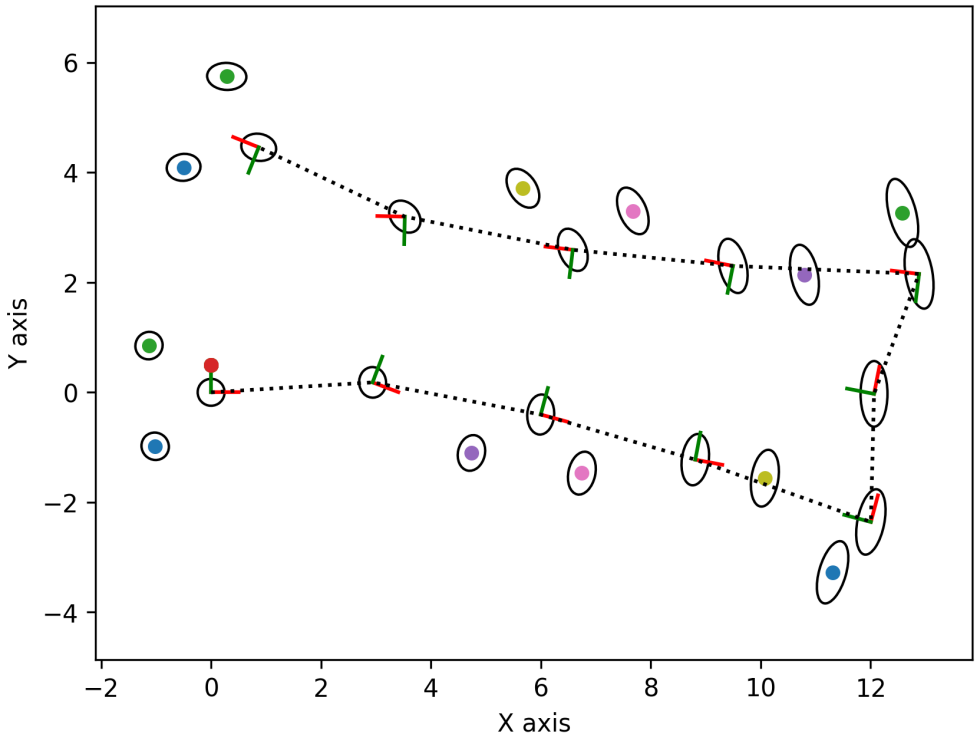


Figure 4: GTSAM Optimized

(f) What do you notice about the orientation of the solution and the shape and orientation of the confidence ellipses? Why does this occur?

Solution: We expect a discussion that includes the following:

- Overall rotation due to uncorrected odometry drift. The whole system is invariant under rotation except for the prior pose.
- The error ellipses are elongated in the $\hat{\theta}$ direction with respect to a polar coordinate system about the origin. Again, this is due to odometry. We reduce the uncertainty in the states adjacent to the loop closure constraint.
- The error ellipses are generally tighter at the start and end. The start is well constrained by the initial pose and the end is well constrained by the landmarks near the initial pose.

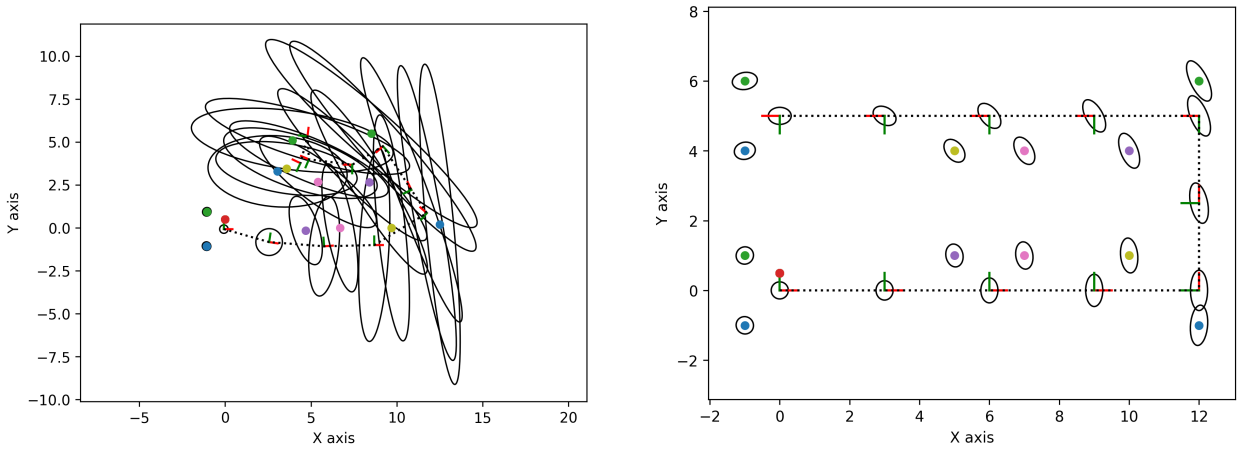
(g) Switch out `robot_history_5.csv` for `robot_history_3.csv` (maximum sensor range of 3 meters) and `robot_history_10.csv` (maximum sensor range of 10 meters). Using the same initialization for `X` and `L`, reconstruct the factor graphs. What are the factor graph sizes in each case?

Solution: We expect 32 factors in the 3-unit range case. We expect 114 factors in the 10-unit range case.

Hint: You should only have to change the file input. All the other functions you wrote should still be applicable!

(h) Plot the optimized estimates for `X` and `L` along with the 2D Gaussian Confidence Ellipse (1-sigma) in each case. What do you notice about the final results at different sensor ranges? Explain your result with the connectivity of the resulting factor graphs.

Solution: The one with a 3-unit range will struggle since the graph is too sparse. The optimizer finds a local minimum and cannot recover. The one with a 10-unit range should look practically perfect since the rover can connect terms across the scene.



(a) 3 unit sensor range

(b) 10 unit sensor range

Figure 5: GTSAM with different sensor range.